

Offline-First Encrypted Content Management System with Conflict-Free Synchronization

Team Members:

1. Gayathri K R
2. Inul Marliya I
3. Divya Dharshana S
4. Jeevika C

Introduction

Content Management Systems are core tools for authoring, organizing, and publishing digital content, but the expectation of always-on connectivity introduces real risks: work interruption, data loss, and synchronization conflicts when multiple devices or users modify content while offline. An offline-first architecture flips that expectation: local edits are authoritative and safely persisted, while the system performs background synchronization when possible. Security must be tightly coupled with this model: sensitive content should remain unreadable on servers and during transit unless the legitimate user provides the cryptographic keys. Our system seeks to combine reliable offline persistence, user-centric encryption, and deterministic conflict resolution so users can continue to author content with confidence regardless of network quality.

Abstract

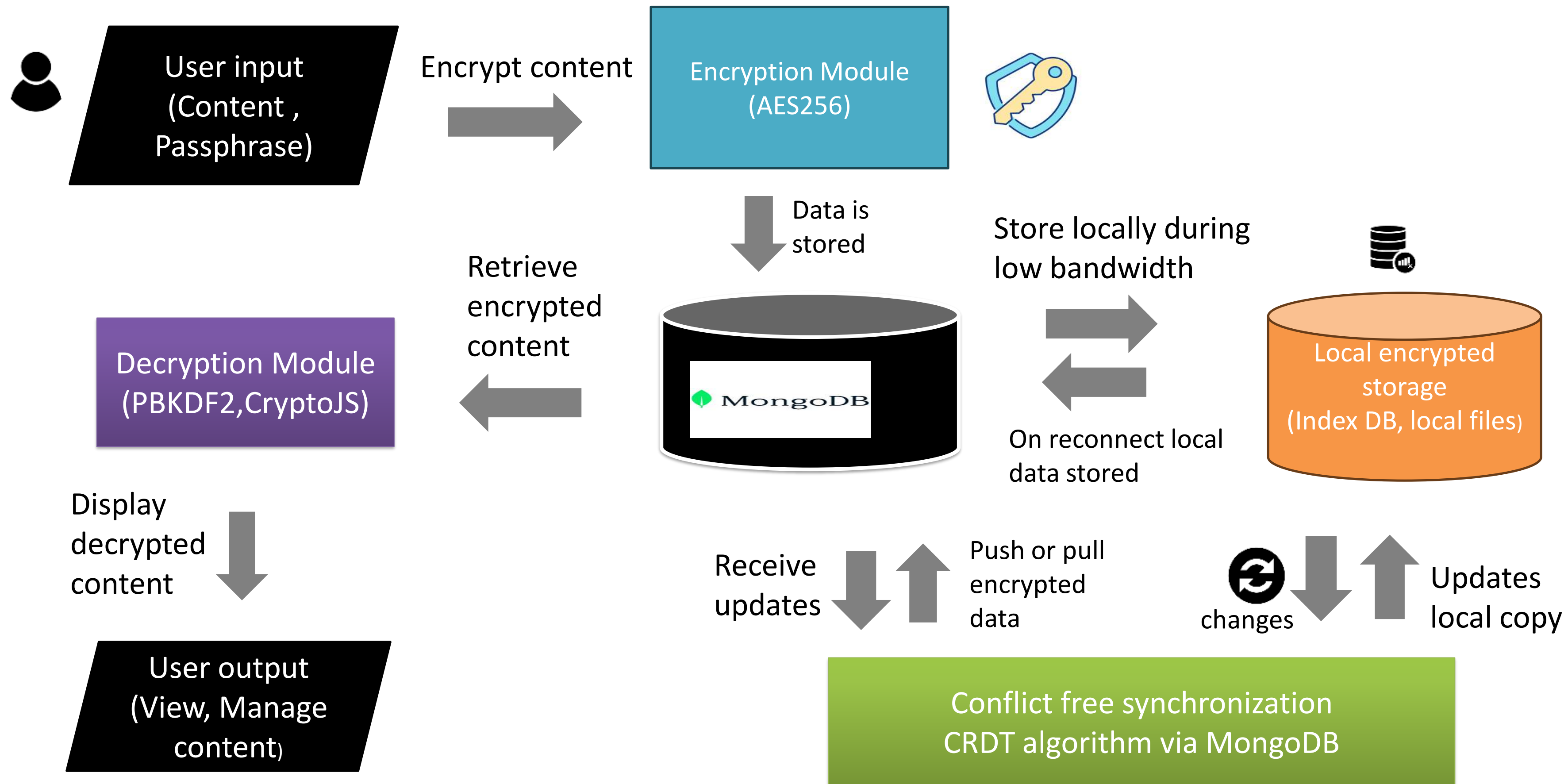
In today's collaborative and mobile-driven world, content creators need tools that remain reliable even without constant internet access. This project presents an offline-first, encrypted Content Management System (CMS) built as a web application using React and a Flask–MongoDB backend. The system supports the creation and management of diverse content types, including text documents, media files, tables, and code snippets. Data is encrypted using AES-256 encryption with a passphrase-based key, ensuring privacy and security. A conflict-free synchronization mechanism based on CRDT (Conflict-free Replicated Data Types) enables seamless multi-user collaboration without overwriting changes. The CMS also supports role-based multi-user access, allowing secure sharing and controlled permissions. Even in the event of lost encryption keys, users can recover their data through secure backup and key-export options. By combining offline capability, robust security, and conflict-safe multi-user sync, this CMS offers a modern and reliable solution for secure, collaborative content management in both connected and disconnected environments.

Existing System

Conventional CMS platforms assume continuous server connectivity and implement versioning and backups server-side; offline editing is either unsupported or delegated to third-party plugins. When offline capabilities exist, synchronization is frequently handled by middleware such as PouchDB/CouchDB or by vendor cloud services, which leaves sensitive content exposed in server plaintext or requires complex server-side encryption schemes. Conflict resolution in these systems often relies on last-write-wins or manual merges, which can lead to lost edits. For enterprises and privacy-sensitive users, this model is unsatisfactory: they need reliable offline editing, robust automatic merging of concurrent changes, and client-side confidentiality with minimal reliance on external services.

Proposed System

We propose an integrated CMS that provides: (1) a PWA frontend that stores edits locally in IndexedDB; (2) client-side encryption where the user supplies a passphrase that is processed with a strong key-derivation function (e.g., Argon2 or PBKDF2) to derive an AES-GCM symmetric key used for all content encryption/decryption; (3) a CRDT-based document model (JSON CRDT) so concurrent updates always converge without conflicts; and (4) a background sync engine that uploads encrypted CRDT deltas and pulls remote deltas to replay against local state. The server stores only encrypted blobs and CRDT metadata — it never holds plaintext or user keys. For usability, the system includes optional passphrase hinting (non-recoverable secret hints) and per-device session caching of derived keys (encrypted by device-local OS stores) so the user isn't asked for the passphrase every moment.



Description

Types of supported content

Text based

Media content, Structured data

Code

Mixed content

User authentication

User logs in with pass phrase.

Encryption key is generated from passphrase
It will be used to encrypt or decrypt the content.

User chooses the type of content they want to manipulate.

The front end loads the editor , input fields for that content type

Content input and formatting

Rich text editing tools

Drag and drop or file picker for media upload

Table editor for entering rows and columns

Language selector and syntax

highlighting for code related content

Encryption and Conflict free Synchronization

AES-256 encryption applied to each content using key from passphrase(PBKDF2). Encrypted content stored locally(IndexedDB) for offline access and MongoDB is the central backend storage to store only encrypted content blobs.

On reconnect , system uses CRDT(Conflict free Replicated Data Type) algorithm to merge changes without overwriting and data loss of edits.

Multi user access handling

Each user has their own encryption key, shared content encrypted with a shared key only for authorised users. Sync engine merges edits from multiple users while preserving changes.

When user views content, it is fetched from local storage or server, decrypted on client side and displayed in correct format.

Conclusion

By combining client-side encryption, an offline-first architecture, and CRDT-based conflict resolution, the proposed CMS achieves three primary goals: continuous availability for authors working in low-connectivity environments, cryptographic privacy so servers cannot read user content, and deterministic, lossless synchronization across devices. This approach reduces risks associated with network outages and third-party synchronization while simplifying the end-user experience: authors can create, edit, and publish content without concern for connectivity or unauthorized data access.

Future Work

- Secure federated servers for cross-organization content sharing.
- Mobile native app clients reusing the CRDT + encryption stack.
- Encrypted version history with searchable encrypted indices.
- Usability research into passphrase management (hardware keys, social recovery).
- Offline AI features (local models) for content suggestions while preserving privacy.

References

- <https://github.com/FieldDB/FieldDB>
- <https://github.com/lybekk/offPIM>
- [https://en.wikipedia.org/wiki/Hoodie_\(software\)](https://en.wikipedia.org/wiki/Hoodie_(software))
- <https://cushiondb.github.io/>
- <https://www.sitepoint.com/offline-web-apps-service-workers-pouchdb/>
- <https://arxiv.org/abs/1608.03960> (Kleppmann & Beresford — Conflict-free replicated JSON datatype)